

Scope and Closures

PREFACE

-A *scope* = *environment* is something functions and expressions are associated with that tells them (functions and expressions) what value each variable refers to

-It is used to figure out which environment are expressions evaluated in.

-same variable name can be used in many places, referring to different things in different locations, but the scope (or *environment*) tells R exactly how to map names to values

- In R, you have both: implicit environments—that I will tend to call scopes—and explicit environments

- The implicit is what you will use in most everyday programming, while the explicit is used in more advanced techniques, which mostly add flexibility → understanding how this works can make it easier to write functions.

-let us start with a simple example:

➔ If you write an expression, such as $x + y$, you cannot evaluate without knowing what x and y refer to. And neither can R

➔ If you have told R what the two variables refer to:

```
x <- 2  
y <- 3  
x + y  
## [1] 5
```

you get what you expect. R knows that x refers to 2 (because you just assigned 2 to it), and it knows that y refers to 3.

-You can think of this as R having a table that maps between names and values

- If we wrote the preceding code at the global (outermost) level ➔the table is the “global environment” or “global scope,” and we can get the table using the `globalenv()` function:

```
genv <- globalenv()
```

-and from it, we can get the values we assigned to the variables x and y:

```
genv$x  
## [1] 2  
  
genv$y  
## [1] 3
```

When we evaluated `x + y`, R automatically looked up the values for x and y, and it looked in the global

environment because we wrote the expression in the global environment.

- We can do the same thing ($x+y$; where x is 2 and y is 3) in a way that makes the environment and evaluation explicit. You can create an expression without associated values using the `quote()` function:

```
expr <- quote(x + y)
```

`expr` refers to the expression $x + y$, but the expression is not evaluated. It is the raw syntax for the addition. To evaluate it, we need the function `eval()`:

```
eval(expr)
```

```
## [1] 5
```

-We get the same value as before because `eval()` also uses the current environment to find the mapping between variable names and values, so `eval(expr)` is short for `eval(expr, genv)` in this case:

```
eval(expr, genv)
```

```
## [1] 5
```

- You can, if you want, create your own environments. The `new.env()` function creates an empty environment that you can then write to:

```
env <- new.env()
```

```
env$x <- 4
```

```
env$y <- 5
eval(expr, env)
## [1] 9
```

or the `list2env()` lets you translate a list into an environment:

```
eval(expr, list2env(list(x = -1, y = -2)))
## [1] -3
```

NB = `list2env()` utility lies in this example :

```
expr <- quote(x + y)
vals <- list(x = -1, y = -2)
eval(expr, vals) # Doesn't work → R expected envirn
not a list of values
e <- list2env(vals)
eval(expr, e)
# -3 → This becomes especially useful when you
already have many variables stored in a list.
```

-When you evaluate an expression such as `x + y`, using the default environment to get the values that the variables refer to, we call it *standard evaluation*.

When you are explicit about the environments and use alternative environments than the default, we call it *nonstandard evaluation*.

- You rarely have to evaluate expressions using explicit environments—although it has its uses—but keep in mind that whenever we evaluate something

where we need to know what variables refer to, some sort of lookup in a table is involved.

Environments and Functions

-It gets more interesting when we add functions. With functions, we have both the global environment and the local environment for a function.

- In the following code, we have two variables named `x`. One is a global variable that refers to a vector. The other (inside the function) is an argument

```
x <- 1:10  
f <- function(x) sqrt(sum(x))  
f(x^2)
```

THE KEY MESSAGE : “When R sees a name like `x`, how does it know which `x` you mean?” → so what happens exactly ? :

- When we evaluate the function → R first create what we call a promise; the promise being `x^2`, which R we look for if its needed inside the function in the calling environment (the global environment).

- when the function call arrives to `sqrt(sum(x))` → "Is there an `x` in the current (function) environment?" →

Yes selon la function x is the main argument ($f(x = x^2)$) → that shows that, R uses that x . It never goes back to the global x .

- Since the global x is $x <- 1:10$ → $f(x^2)$ → $f(x = c(1,4,9,16,25,36,49,64,81,100))$ → So it uses that x

NB : $x <- 10$

```
f <- function(x) { x + 1 }
```

$f(5)$ → here the value has already been given

However , Using the same name in the original example can make it seem like there's only one x , but there are really two variables living in different environments. R determines which one to use based on where the expression is being evaluated.

We can clarify this by changing the names so the variables become unique. We call the global variable gx and the parameter variable px :

```
gx <- 1:10
```

```
f <- function(px) sqrt(sum(px))
```

$f(gx^2)$ → run it → same results

- Since the global gx is $gx <- 1:10$ → $f(gx^2)$ → $f(gx^2 = c(1,4,9,16,25,36,49,64,81,100))$

- When we evaluate the function → R first create what we call a promess; the promess being gx^2

- when the function call arrives to `sqrt(sum(px))` → R say "Is there a `px` in the current (function) environment?" → Yes selon la fonction `px` is the main argument (`f(px = gx^2)`) → that shows that, R uses that `px`. It never goes back to the global `x`.

Key message : The repeated name `x` in the first version is just a coincidence. There are actually two independent variables:

- `gx` (global variable)
- `px` (function parameter)

we can try to see the local enviro inside the function :

```
gx <- 1:10
f <- function(px) as.list(environment())
f(gx**2)
f(gx)
$px
[1] 1 2 3 4 5 6 7 8 9 10
```

-Generally, the local environment of a function call will have the parameters and any variable you assign inside the function:

```
f <- function(x) {
  res <- sqrt(sum(x))
  as.list(environment())
}
```

```

}
x <- 1:10
f(x**2)

f(x**2)

$res

[1] 19.62142

$x

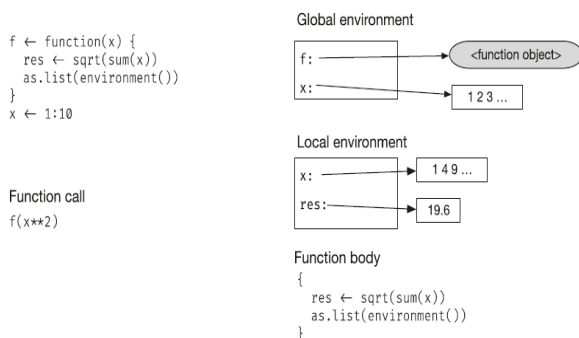
[1] 1 4 9 16 25 36 49 64 81 100

```

Because I assigned to res, that variable is now also in the local environment.

global x and a local x do not clash. the local environment that contains the local variables and the global environment that includes the global variables; see Figure [bellow : explain in details the figure bellow:](#)

Picture 1



You might now wonder about the names `sqrt()` and `sum()` that the function `f` also seems to know. They aren't in the local environment. (They aren't in the

global environment either; you can check with `as.list(globalenv()).`)

So how does the function find them? We shall now see that there is one more trick to environments that handles finding variables that are not in the immediate scope and how that trick solves this problem in the section.

Environment Chains, Scope, and Function Calls

-When you call a function in R, you first create an empty environment for that function. That environment is where its parameters will be stored, and any local variables the function assigns to will also go there

-Whenever R needs to look up a value, it will look in this environment. If it finds the variable, you get the value in this table (table is the environment just for exemple to understand) → If it doesn't, R doesn't just give up, however. It continues the search.

-Environments are not only table but chains of tables

-Each environment has a "parent" (you can use the `parent.env()` function to get it), and if a lookup fails for an environment, R will look in the parent instead. If it fails there, it will try the grandparent, and it will continue as long as there are environments.

Ex 1 :

```
e1 <- new.env()
```

```
e2 <- new.env(parent = e1)
```

```
parent.env(e2)
```

we can verify it → identical(parent.env(e2), e1) # TRUE

Ex 2 :

as we said is R does find a variable in the actual environment, R will look in parent environ and will still look until the last environment :for exemple

```
e1 <- new.env()
```

```
e1$x <- 10
```

```
as.list(e1)
```

```
e2 <- new.env(parent = e1)
```

```
as.list(e2)
```

```
parent.env(e2)
```

e2\$x # wont work wont give 10 because dollar sign does not search parent environments instead we do :

get("x", envir = e2, inherits = TRUE) → alternative :

exists("x", envir = e2, inherits = TRUE) or

eval(quote(x), envir = e2)

Ex 3 : we gonna see here whats happens when we activate a function....step by step....

```
f <- function(x) {  
  sqrt(sum(x + y))  
}  
x <- 1:10  
y <- 1:10  
f(x**2)
```

[1] 20.97618

We define three variables, f, x, and y, in the global environment, where f is a function. Then we call the function, creating a local environment where f's argument x will be placed; see Figure bellow :

Picture 2

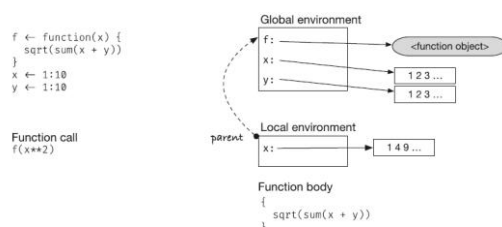
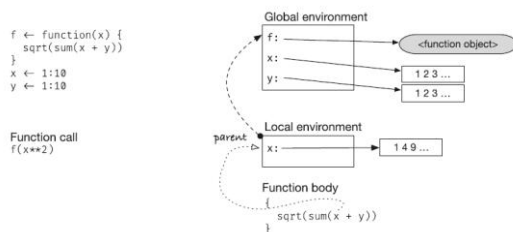


Figure 4-2. The local and global environments are connected through a parent pointer

The x in the local and the global environments refers to different values, and if we look for the value of x inside the function, we get the one in the local environment; see Figure bellow :

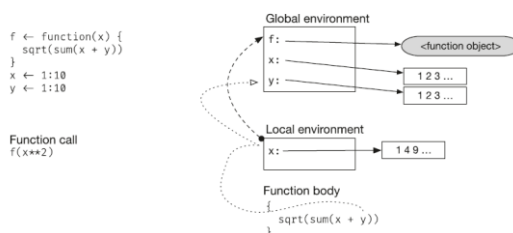
Picture 3



The `sqrt`, `sum`, and `y` names used in the function body are not defined in the local environment

The parent environment, however, allows us to find nonlocal variables. When R cannot find `y` in the local environment, it will go to the parent environment, which is the global environment, and find it there

Picture 4



the environment chain goes deeper than that. After the global environment, you will find environments that hold names from loaded packages, which is why you can use functions from packages → the so-called “base environment,” where you find the base R functions (you can get hold of it with the `baseenv()` function :

`ls(baseenv())`

`ls(baseenv())[1:10]`

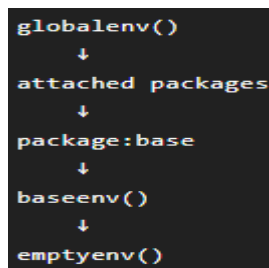
`baseenv()$mean`

`baseenv()$sum`

`baseenv()$print`

The environment hierarchy looks roughly like this:

Picture 13



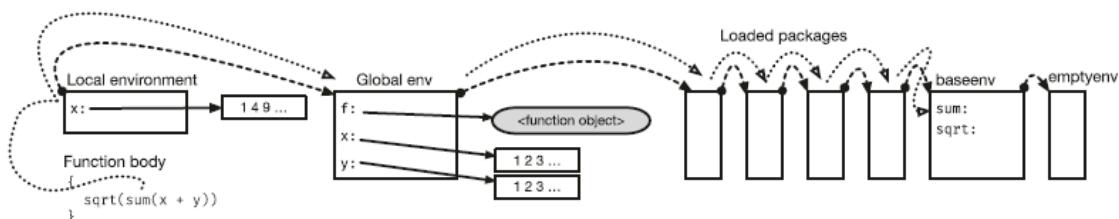
`parent.env(baseenv())`

`ls(emptyenv())` is really empty...no parent envi... : if we try :

`parent.env(emptyenv())` → Error in
`parent.env(emptyenv())` : the empty environment has no parent

another hierarchy image :

Picture 5



Whenever you need the value of a variable, whether you use standard evaluation where R finds the environment for you or you provide an environment to eval in a nonstandard evaluation, you will use the chain of parent environments to find variables that are not in the immediate scope.

Scopes, Lazy Evaluation, and Default Parameters

Knowing the rules for how variables are mapped into values also helps us understand why we can use function parameters in expressions for default parameters when defining a function, but we cannot use them when we call a function.==> better explained with PW (pen windows)

First cas study = *Evaluating with an default argument*

If we define a function with a default parameter set to an expression that refers to another parameter

```
f <- function(x, y = 2 * x) x + y
```

we can call it like this:

```
a <- 2  
f(x = a)  
## [1] 6
```

but not necessarily like this:

```
f(x = a, y = 2 * x)
```

In both cases, we will create an environment for the function first, and in it, we will store two promises → using lazy evaluation. You can think of this lazy evaluation much like the `quote(x + y)` → It is unevaluated expressions we pass to the function, and we only evaluate them once we have to.

f's environment

`x` → promise: `a` `y` → promise: `2 * x`

Then the body: `x + y` → forces both promises.

```
f <- function(x, y = 2 * x) x + y
```

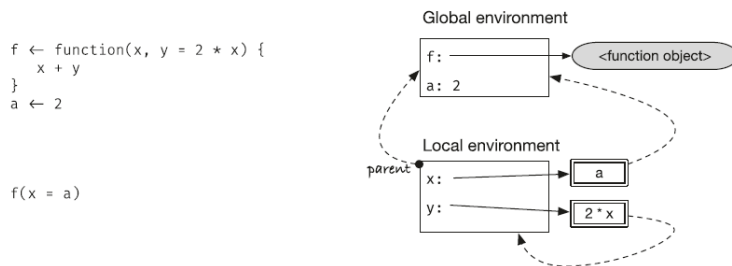
```
a <- 2
```

```
f(x = a)
```

```
## [1] 6
```

first case, where `x` is provided but `y` is the default, the expression `a` for `x` knows that it should be evaluated in the global scope, where `a` is defined, but the default parameter `y = 2 * x` should be evaluated in the local environment → meaning the `x` inside `y = 2 * x` will be looked for in local enviro not en the global

Picture 6



we can see it more if we do:

```
f <- function(x, y = 2 * x) x + y
```

`f(x = 2a)` → `y` is the default + `x = 2a` → when the function is called (used) by activating `f(x = 2a)` → `x` and `y` are promises (not evaluated until they are needed) → when we are in the `x+y` line → `x` will be evaluated in global enviro because in the function call `f(x=2a)` / and `y` will be evaluated inside function call → meaning `y` will be `y = 2*x` → and the `x` is not the one in the global environ instead it's the `x = 2a`

the `x` in the inside is not `x` in the global envi...the `x` in the global is 2 and `x` inside is `2*a` which is 4

second case study : Evaluating with an explicit argument

```
f <- function(x, y = 2 * x) x + y
```

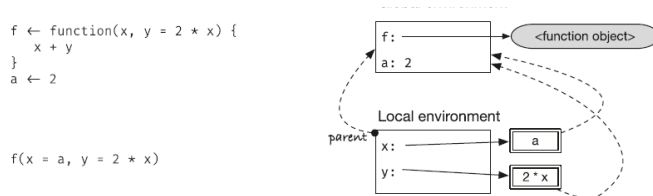
```
a <- 2
```

```
f(x = a, y = 2*x)
```

the same steps as before when we call the function → promises until used → when we arrive in the `x+y` in this

case, x will be x in the global environ and x inside the $y = 2 * x$ will also be in the global environ

Picture 7



We can exploit lazy evaluation to extract expressions rather than values and evaluate them in different environments, should we choose. A full exploration of this is beyond the scope of this book, but a short example could look like this:

```
expr <- quote(x + 2)
```

```
str(expr)
```

```
x <- 3 ➔ global enviro
```

```
e <- new.env()
```

```
e$x <- 100
```

```
x+2
```

`eval(expr , e)` => works with an expression object that you already have.

we can make a function that deals a function argument as an expression.

But we can imagine that the function will take more than that.... for example the function argument can be like `f(x+y+z, x=1,y=2)` or maybe more `f(x+y+z+w, x=1,y=2,z=5)`....so the first argument may be exprs and the other should be capable to incorporating a lot of possibilities...the most appropriate is "...". Here is the function:

```
f <- function (exprs,...) {  
  }  
}
```

We must deal with the `exprs` argument → in a sense where `eval` function extract express by using `quote`.

`Quote` wont do the trick here because as we already said `eval(expr , e)=>` works with an expression object that you already have not with function argument. To deal or to extract `exprs` from function argument we use `substitute ()` function:

```
f <- function (exprs, ...) {  
  expr <- substitute(expr)  
}
```

And then we would want to put the variable that's gonna be in "..." in list (list can contain anything) for example.

```
f <- function (exprs, ...) {  
  expr <- substitute(expr)  
  list(...)  
}
```

And then put the variable that's inside the list in env.
There's already function in base R that does exactly that:

```
f <- function (exprs, ...) {  
  expr <- substitute(expr)  
  lt <- list(...)  
  env <- list2env(lt)  
}
```

And then we could just add the eval function to evaluate the exprs inside the env newly created

```
f <- function (expr, ...) {  
  expr <- substitute(expr)  
  lt <- list(...)  
  env <- list2env(lt)  
  eval(expr, env)  
}
```

We going to test the code:

```
x <- 10
```

```
f(x + y, y = 2)
```

```
[1] 12 ➔ works
```

```
a <- 22
```

```
f(a)
```

```
## [1] 22
```

```
f(a + b, b = 13)
```

```
## [1] 35
```

Small word about that in the book they added another function `parent.frame()` because if not added R throws an error about not finding `y`. but now, the function is not needed, probably because has done updates to fix this.

```
f <- function (expr, ...) {
```

```
  expr <- substitute(expr)
```

```
  lt <- list(...)
```

```
  env <- list2env(lt)
```

```
  parent.env(env) <- parent.frame()
```

```
  eval(expr, env)
```

```
}
```

Nested Functions and Scopes

So far, we have only considered two environments—global and local—so it might be natural to assume that the parent of a local environment is always the global → but this isn't the case.

Consider this slightly weird but simple enough function:

```
f <- function() environment()
```

When we call `f()`, it will return its local environment, and there is nothing we haven't seen before yet.

```
f()
```

```
## <environment: 0x7fe46613ae88>
```

However, the `environment()` function takes an argument, a function, and if you call it with a function, it returns something else than the current environment. It returns the environment where the function was defined → We defined `f` in the global environment, so `environment(f)` will give us the global environment:

```
environment(f)
```

```
## <environment: R_GlobalEnv>
```

which, as should be clear by now, is different from the environment we get when we are inside the function's body:

```
f()
```

```
## <environment: 0x7fe46613ae88>
```

To expand on this a little, let us try to define a function inside f:

```
f <- function(x) {  
  print(environment()) # Print the local environment  
  g <- function(y) x + y  
  g  
}
```

Since f is still defined in the global environment, environment(f) is still R_GlobalEnv :

```
environment(f)
```

but the function g we define inside a call to f is defined in the local environment:

```
g <- f(2)
```

```
<environment: 0x0000026a37ec1b60>
```

```
environment(g)
```

```
<environment: 0x0000026a37ec1b60>
```

As we see, the environment that we print inside the call to f() is the same as the environment associated with the function we get as the result of the function call.

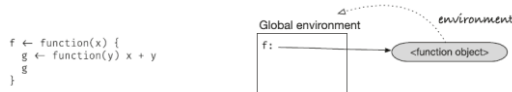
The function's environment is an existing environment where we have defined the function, while the local environment is a new environment created when we call the function ➔ If that is clear, then the rule for how

R sets up the parent environment is simple enough → when you call a function, the new local environment will get the function's environment as its parent.

Let's go through how the environment are set up in the preceding example in detail:

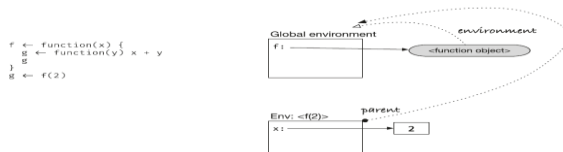
We start right after we have defined `f` → In the global environment, we have a reference to the new function—the variable `f` refers to the function object—and the function's environment is the global environment (where we defined it)

Picture 8



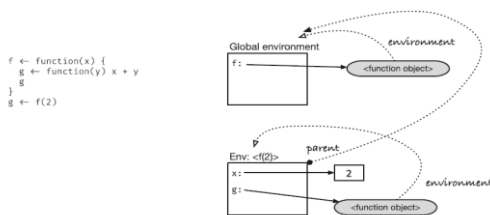
When we call `f` (activation of `f(2)`), we create a new environment for the function call, the parent of the new environment is the function's environment (the global environment), and we put the argument `x` into the local environment

Picture 9



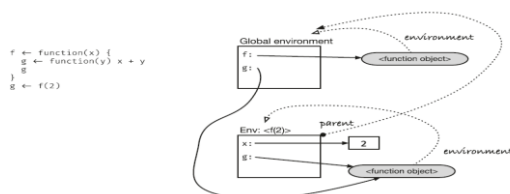
Then, inside the function call, we define the function **g** → That creates a new function object, and since it is defined inside the function call environment, that environment becomes **g**'s environment

Picture 10



We return the new function object from the call to **f**, and in this code, we assign it to a variable **g** in the global environment

Picture 11



We didn't have to name the function object the same in the global and local environments, but in this code, we use the variable **g** in both environments to refer to the same object.

Usually, you can think of the local environment of a function call as disappearing when you return from a call because it is inaccessible after the function returns, but not in this case. The function we call **g** in the global

environment has the `f(2)` call's environment as its function environment, so the environment is still around, which is important if we now call `g` (from the global environment).

Let's see what happens in this call. You can follow along below :

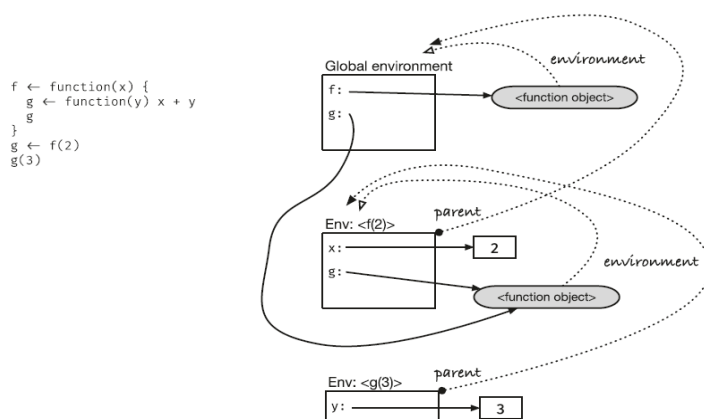
`g(3)`

[1] 5

We create a new environment for the call `g(3)`, and since `g`'s environment is the one we created when we called `f(2)`, that will be the parent environment.

The parameter `y` is inserted in the new environment, so when we evaluate `x + y`, we can get `y` from there. The variable `x` is not in the local environment, so we must follow the parent pointer to the `f(2)` environment, where we find that it maps to 2.

Picture 12



Closures

The `g` function we have written here is an example of a so-called *closure* → a function that can find some of the variables it needs in the environment of the function call that created it.

```
f <- function(x) {  
  print(environment()) # Print the local environment  
  g <- function(y) x + y  
  g  
}
```

```
g<- f(2)
```

```
g(y = 3)
```

With closures, we can adapt the same function to different data by putting it in different environment chains. With the `f` from before, we could write:

```
h1 <- f(1)
```

```
## <environment: 0x7fe446992678>
```

```
h2 <- f(2)
```

```
## <environment: 0x7fe4777ada20>
```

```
h3 <- f(3)
```

```
## <environment: 0x7fe477056128>
```

We get three different functions (`h1`, `h2`, `h3`), one that adds 1 to its input (because `g <- function(y) x + y` and

$f(1)$ means $f(x=1) \rightarrow$ and inside f there g and inside g there is $x+y \rightarrow 1+y$, one that adds 2 ($2+y$), and one that adds 3 ($3+y$), all from the same code. The variable x will be found in three different environments—environments created when we called f and that map x to 1, 2, or 3, respectively:

`sapply(1:5, h1)` $\rightarrow$ its we say apply $h1$ to serie of numbers from 1 to 5 $\rightarrow h1(1)$ give result and then $h1(2)$

$h1$ as we already just said that $h1$ wil give $\rightarrow 1+y$ and with `sapply` $\rightarrow x$ will be from 1 to 5 $\rightarrow (1 \text{ to } 5)+1$

```
## [1] 2 3 4 5 6
```

```
sapply(1:5, h2)
```

```
## [1] 3 4 5 6 7
```

```
sapply(1:5, h3)
```

```
## [1] 4 5 6 7 8
```

Closures are frequently used in functional programming because $\rightarrow$ we can separate domain information from generic code. Look in `sapply` exemple $\rightarrow$ the $h1$ function already store in its envirimnt some variable $\rightarrow$ and uses the sequence 1 to 5 in this exemple to give results

Reaching Outside Your Innermost Scope (use PW)

When we assign to a variable using the `<-` operator inside a function, we modify the environment at the top of the current environment chain. We modify the local environment ➔

Ex1:

```
x <- 1
```

```
f <- function() {
```

```
  x <- 10
```

```
  print(x)
```

```
}
```

```
x <- 1
```

```
f <- function() {
```

```
  x <- 10
```

```
  print(x)
```

```
}
```

```
f() # 10
```

```
x # 1
```

Ex2:

We want to create a function that returns an increasing number each time we call it.

```
make_counter <- function() {  
  x <- 0  
  count <- function() {  
    x <- x + 1  
    x  
  }  
}  
counter <- make_counter()
```

counter() → doesn't work will always give 111

We can unwrap the function and see what is going on. When we create the counter, we call the make_counter function, which creates an environment where x is set to zero, and then the count function, which it returns → When we call the counter function, it knows x because it is a closure, so it can evaluate x + 1, which it then assigns to x → This is where the problem is → The x used in evaluating x + 1 is found by searching up the environment chain, but the x the function assigns to is put in the counter function's environment → Then the function returns, and that environment is gone → The next time counter is called, it is a new function instance, creating a new environment. The variable x from the make_counter function is never updated.

you need to use the operator `<<-` instead➡ this operator will search through the environment chain➡ and update the environment where it finds the variable (or add it to the global environment if it doesn't find it anywhere in the environment chain).

We going to make a function using a tree as an example + counter + `<<-`. This example is little bit complex but I will try to make look easy...

Suppose we had a tree like this one :

```
      Root (1,3)
     /   \
  C(1,2) D(3,3)
   / \
A(1,1) B(2,2)
```

```
make_node <- function(name , left = NULL , right =
NULL) {
  list(name = name, left = left , right = right)
}
```

```

tree <- make_node(name = "root",
                  left = make_node(name = "C",
                                   left = make_node("A", left = NULL,
                                                    right = NULL),
                                   right = make_node("B", left = NULL,
                                                    , right = NULL)),
                  right = make_node("D", left = NULL, right =
NULL))

```

```

make_counter <- function() {
  x <- 0
  count <- function() {
    x <-- x + 1
    x
  }
}

```

We want our function to do two things: 1-to give the leaf nodes depth-first numbers: A = 1 B = 2 D = 3 and then calculate a range for every node. For example: root range is 1 to 3 , c : 1 to 2.

So starting with first objective (leaf nodes depth numbers) ➔ first we should figure out how are will

detect leafleaf if you think about it are just leaf without children.... So leaf is left = NULL and right = NULL in the same time. So :

```
depth_first_numbers <- function(tree) {
```

```
  if (is.null(tree$left ) && is.null(tree$right)) {  && is  
  used for single values and & vector comparison
```

```
  }
```

```
}
```

We want to count if is.null(tree\$left) && is.null(tree\$right) are true. To do so we use the counter function:

```
depth_first_numbers <- function(tree) {
```

```
  if (is.null(tree$left ) && is.null(tree$right)) {
```

```
    df <- counter()
```

```
  }
```

```
}
```

And we can say that the df variable we want to put in table where every time its activated will be associated with the current leaf like this :

```

depth_first_numbers <- function(tree) {
  if (is.null(tree$left ) && is.null(tree$right)) {
    df <- counter()
    table <- df
  }
}

```

To precise we need to add something to the table variable...where we want df to be put inside the table :

```

depth_first_numbers <- function(tree) {
  if (is.null(tree$left ) && is.null(tree$right)) {
    df <- counter()
    table[tree$name] <- df
  }
}

```

We need two things ➔ make table where df and leaf name will be put on and also the counter function

```

depth_first_numbers <- function(tree) {
  counter <- make_counter()
  table <- c()
  if (is.null(tree$left ) && is.null(tree$right)) {
    df <- counter()
    table[tree$name] <- df
  }
}

```

Now we gonna add the range variable → its new variable that going to be like we said [1,1] or 2,2 or 3,3 → because leaf with not children :

```

depth_first_numbers <- function(tree) {
  counter <- make_counter()
  table <- c()
  if (is.null(tree$left ) && is.null(tree$right)) {
    df <- counter()
    tree$range <- c(df,df)
    table[tree$name] <- df
  }
}

```

```
}
```

That the first part. We need to add the else part....if leaf had children....but before thatwe need to add line so R will use that last line in the else part....

```
depth_first_numbers <- function(tree) {  
  counter <- make_counter()  
  table <- c()  
  if (is.null(tree$left ) && is.null(tree$right)) {  
    df <- counter()  
    tree $range <- c(df,df)  
    table[tree$name] <- df  
    tree  
  } else {  
  }  
}
```

So the first if part...will only look for leaf = nodes without children.....we need to see what to do about the one who have children....like before we going to use recursive function to transverse all the tree....

```

depth_first_numbers <- function(tree) {
  counter <- make_counter()
  table <- c()
  if (is.null(tree$left ) && is.null(tree$right)) {
    df <- counter()
    tree $range <- c(df,df)
    table[tree$name] <- df
    tree
  } else {
    Left <- depth_first_numbers(tree$left)
    Right <- depth_first_numbers(tree$right)
  }
}

```

There is something off about the functionin the else part....we used depth_first_numbers recursively but what this mean.....its means is that R will go again in counter <- make_counter()

table <- c() this will create new data , new counter and we don't want that....so we need to do something to separate the two actions....the best way its to give

every action a name like this ...or in this situation since we already have `depth_first_numbers`...we are gonna give the if part another named function :

```
depth_first_numbers <- function(tree) {  
  
  counter <- make_counter()  
  
  table <- c()  
  
  traverse_tree <- function(node) {  
  
    if (is.null(node $left ) && is.null(node $right)) {  
  
      df <- counter()  
  
      node $range <- c(df,df)  
  
      table[node $name] <- df  
  
      node  
  
    } else {  
  
      Left <- depth_first_numbers(node $left)  
  
      Right <- depth_first_numbers(node $right)  
  
    }  
  
  }  
  
}
```

And also the we should adapt the recursive call name from `depth_firstnumber` to `traverse_tree` function :

```
depth_first_numbers <- function(tree) {  
  counter <- make_counter()  
  table <- c()  
  traverse_tree <- function(node) {  
    if (is.null(node $left ) && is.null(node $right)) {  
      df <- counter()  
      node $range <- c(df,df)  
      table[node $name] <- df  
      node  
    } else {  
      Left <- traverse_tree (node $left)  
      Right <- traverse_tree (node $right)  
    }  
  }  
}
```

And its better if we use another argument for the new function just to make things simple and clear and

adapt the code about the new argument added : node instead of tree

```
depth_first_numbers <- function(tree) {  
  
  counter <- make_counter()  
  
  table <- c()  
  
  traverse_tree <- function(node) {  
  
    if (is.null(node$left ) && is.null(node$right)) {  
  
      df <- counter()  
  
      node $range <- c(df,df)  
  
      table[node$name] <- df  
  
      node  
  
    } else {  
  
      Left <- traverse_tree (node$left)  
  
      Right <- traverse_tree (node$right)  
  
    }  
  
  }  
  
}
```

Then we continue with the else partin the if part we did make a table and store the result in table but

range variable we want also to print it in the tree to do so we are going to need to add it manually like so :

```
depth_first_numbers <- function(tree) {  
  
  counter <- make_counter()  
  
  table <- c()  
  
  traverse_tree <- function(node) {  
  
    if (is.null(node $left ) && is.null(node $right)) {  
  
      df <- counter()  
  
      node $range <- c(df,df)  
  
      table[node$name] <- df  
  
      node  
  
    } else {  
  
      Left <- traverse_tree (node $left)  
  
      Right <- traverse_tree (node $right)  
  
      New_node <- make_node(node$name,left,right)  
  
      New_node$range <- c( left$range[1], right$range[2]  
      )  
  
    }  
  
  }  
}
```

**New_node\$range <- c(left\$range[1], right\$range[2]
)→ its gonna take the the “tree” part in the last line of
if part.....that contain the node\$range dataand
combine theme in the range variable....and of course
we should always finish with the latst data update
which is new_node**

depth_first_numbers <- function(tree) {

counter <- make_counter()

table <- c()

traverse_tree <- function(node) {

if (is.null(node \$left) && is.null(node \$right)) {

df <- counter()

node \$range <- c(df,df)

table[node \$name] <- df

node } else {

Left <- traverse_tree (node \$left)

Right <- traverse_tree (node \$right)

New_node <- make_node(node \$name,left,right)

New_node\$range <- c(left\$range[1], right\$range[2])

New_node}}}

If we use the function :

`depth_first_numbers(tree)` → nothing shows → something is missingsee that the function first if and the else part have been updated separately.....we need to add something so that both the first and last part be printed we can try and add tree !!... and the tree should be put before the end but after the two part (if and else) have been evaluated:

```
depth_first_numbers <- function(tree) {  
  counter <- make_counter()  
  table <- c()  
  traverse_tree <- function(node) {  
    if (is.null(node $left ) && is.null(node $right)) {  
      df <- counter()  
      node $range <- c(df,df)  
      table[node $name] <- df  
      node } else {  
        Left <- traverse_tree (node $left)  
        Right <- traverse_tree (node $right)  
        New_node <- make_node(node $name,left,right)
```

```
New_node$range <- c( left$range[1], right$range[2]
)
```

```
New_node
```

```
}
```

```
}
```

```
tree
```

```
}
```

depth_first_numbers(tree) → give only the tree without range nor tableso we need the updated version to do so :

```
depth_first_numbers <- function(tree) {
```

```
  counter <- make_counter()
```

```
  table <- c()
```

```
  traverse_tree <- function(node) {
```

```
    if (is.null(node $left ) && is.null(node $right)) {
```

```
      df <- counter()
```

```
      node $range <- c(df,df)
```

```
      table[node $name] <- df
```

```
    node } else {
```

```
Left <- traverse_tree (node $left)
```

```
Right <- traverse_tree (node $right)
```

```
New_node <- make_node(node $name,left,right)
```

```
New_node$range <- c( left$range[1], right$range[2]  
)
```

```
New_node
```

```
}
```

```
}
```

```
new_tree <- traverse_tree(tree)
```

```
new_tree
```

```
}
```

depth_first_numbers(tree)=→ good tree with the rangethe onlu thing missing is table we can add it. And the best variable where to put tree and table together is list like so :

```
depth_first_numbers <- function(tree) {
```

```
  counter <- make_counter()
```

```
  table <- c()
```

```
  traverse_tree <- function(node) {
```

```
    if (is.null(node $left ) && is.null(node $right)) {
```

```

df <- counter()
node $range <- c(df,df)
table[node $name] <- df
node } else {
Left <- traverse_tree (node $left)
Right <- traverse_tree (node $right)
New_node <- make_node(node $name,left,right)
New_node$range <- c( left$range[1], right$range[2]
)
New_node
}
}

new_tree <- traverse_tree(tree)
list(tree = new_tree, table = table)
}

depth_first_numbers(tree)→ its give NULL in the table
part. So R is not storing what we send in table variable
we need to add the <<-

depth_first_numbers <- function(tree) {

```

```

counter <- make_counter()

table <- c()

traverse_tree <- function(node) {
  if (is.null(node $left ) && is.null(node $right)) {
    df <- counter()
    node $range <- c(df,df)
    table[node $name] <- df
  } else {
    Left <- traverse_tree (node $left)
    Right <- traverse_tree (node $right)
    New_node <- make_node(node $name,left,right)
    New_node$range <- c( left$range[1], right$range[2]
    )
    New_node
  }
}

new_tree <- traverse_tree(tree)
list(tree = new_tree, table = table)
}

```

`depth_first_numbers(tree)` → its workshere are the ranges and the table.

Lexical and Dynamic Scope

-This rule for finding variable values, based on the environment where functions are defined, is called *lexical scoping* and is the most common standard for scopes → They are called that because you can, in principle, figure out what variables in an expression are defined where → You first check if the variables are set in the local environment, either local variables or function parameters. If not, you look at the enclosing code and check if the variable is in the environment. The enclosing code can be the global environment or a function. If it is a function and the variables are not defined, you work your way further out in the code where the function is defined

-If the mapping from variables to values depended on where a function was *called* rather than where it was defined, something called *dynamic scope*

-R supports both kinds of scope → lexical scope being the default (standard evaluation) → but you should stick to standard lexical scope in the majority of your projects, as it is by far the easiest to work with.

-One common case where dynamic scope is worth the effort is when you need to evaluate expressions where

some of your data is in variables and some in a data frame. Here simple example of what I am trying to say :

```
x <- 10
```

```
df <- data.frame(  
  x = c(1, 2, 3),  
  y = c(4, 5, 6)  
)
```

If you evaluate an expression inside the data frame:

```
with(df, x + y)
```

R looks for `x` and `y` in the data frame first, producing:
[1] 5 7 9

If you instead use a variable that isn't in the data frame:

```
z <- 100
```

```
with(df, y + z) ➔ [1] 104 105 106
```

dynamic scope is useful here because without this mechanism, you would have to write something like:

```
df$y + z
```

or

```
df$x + df$y
```

now we want to make a function that resembles the `with()` function we just used → and its gonna resemble the one we already made :

```
f <- function(expr, ...) {  
  expr <- substitute(expr)  
  env <- list2env(list(...))  
  eval(expr, env)  
}
```

So are going to try to adjust it → first thing the arguments → we want to try to use expression to do some operation with something inside data frame for example → so we will have two arguments :

```
f <- function(expr, dt) {  
  expr <- substitute(expr)  
  env <- list2env(list(...))  
  eval(expr, env)  
}
```

We will leave as is the `expr` part → because that what we want If you remember the example we mentioned earlier → `with(df, x + y)` → `x+y` is the expression and `df` is the data frame → and the expression we want it to be not evaluated and only the terme = the expression that's why we use `substitute` to get what we want.

```
f <- function(expr, dt) {  
  expr <- substitute(expr)
```

```
env <- list2env(list(...))  
eval(expr, env)  
}
```

Second line will leave it too with small changes...we leave it because we need an environment where we going to evaluated the expression and the data frame.... best choice is to drag the data frame with its data to an environment

```
f <- function(expr, dt) {  
  expr <- substitute(expr)  
  env <- list2env(dt)  
  eval(expr, env)  
}
```

We gonna test it and compare if its behaves like with() function :

f(x+y, df) → works good

but now we are gonna try to use variable not in the data frame

z <- 100

f(y + z, df) → works

but in this case the making f function is just to enhance the utility to think about functions because with() is doing good job....but maybe be if we ware working with huge package where function are complex and lot of data.frames we would want something simple and to the pointthat where f function f will show its utility.